

Leant2: Answers to the Expert Questions

A literature-grounded design for dependent program synthesis,
search control, auxiliary-state invention, and certification

Research and design assessment
Prepared for Vladimir Reshetnikov

September 21, 2026

Abstract

This article answers all 31 questions in the nine research sections of *Leant2: Design for Further Improvements*. It examines the proposal and selected implementation modules at repository commit [784664ff34e8](#), checks relevant primary literature, and turns the questions into explicit algorithms, correctness obligations, and experiments. Several questions already have stronger published answers than the proposal acknowledges: Smyth supports higher-order examples and incomplete recursive examples; Hoogler handles type classes; auxiliary-state synthesis has concrete precedents in AutoLifter, Para, and auxiliary-function synthesis; and Lean exposes reusable motive-construction operations. Other questions need their premises tightened. In particular, summed goal-count heuristics are not necessarily admissible, finite observations do not identify a minimal carrier type, proof irrelevance does not license arbitrary cast erasure, and a hard wall-clock cutoff cannot generally preserve identical candidate sets across machines. The proposed architecture separates certified acceptance, conservative pruning, and heuristic guidance. It combines dependency-aware persistent search, typed residual observations, bounded lifting templates, proof-carrying recursion, and replayable decision traces. Mathematical arguments and a small executable finite-model suite support the central claims; no new Leant2 performance measurements or Lean-checked implementation are claimed.

Reading guide. Sections 1–2 state the findings and evidence boundary. Sections 4–12 answer the questions individually. Section 13 gives the implementation plan; Section 14 specifies how to test it. Appendix A maps every question to its answer.

Contents

1	Scope, evidence, and the main recommendation	4
1.1	Kinds of conclusion used in this article	4
2	Corrections that change the plan	4
3	The semantic contract of the search	5
4	R1: Search control with coupled goals	6
4.1	R1.Q1. Can shared-metavariable goals be organized without wholesale copying or sequen- tialization?	6
4.2	R1.Q2. What admissible heuristic remains valid when unification closes several goals? . . .	7
4.3	R1.Q3. Can just-in-time learning preserve meaningful ranking?	7
4.4	R1.Q4. Can a hard wall-clock budget give machine-independent results?	8
5	R2: Carriers, accumulators, and auxiliary state	8
5.1	R2.Q1. Can finite examples determine a minimal carrier type, or prove that none exists? . .	9
5.2	R2.Q2. What replaces an alphabet for polymorphic functions?	10
5.3	R2.Q3. Can fusion run backward to invent auxiliary information?	10
5.4	R2.Q4. How can motives for primitive recursion with parameters be enumerated?	11
5.5	R2.Q5. Do induction-generalization heuristics transfer to carrier selection?	12
6	R3: Evaluation of programs with holes	12
6.1	R3.Q1. What is already available for higher-order and dependent examples?	12
6.2	R3.Q2. How should normalization caches survive assignment and rollback?	13
6.3	R3.Q3. What is the cheapest route to a provably conservative fast evaluator?	14
6.4	R3.Q4. Can top-down angelic recursion have termination and completeness guarantees? . .	14
7	R4: Indexed families and motive search	15
7.1	R4.Q1. Can a useful motive class be enumerated completely and terminate?	15
7.2	R4.Q2. How should transports be represented without destroying canonical enumeration? . .	15
7.3	R4.Q3. What unfolding strategy is supported by existing experience?	16
7.4	R4.Q4. Can Lean’s motive computation be called directly with useful failures?	16
8	R5: General recursion from incomplete examples	17
8.1	R5.Q1. Does a capability interpreter agree with the realized recursive term definitionally? .	17
8.2	R5.Q2. What changes when angelic execution is combined with top-down search?	18
8.3	R5.Q3. Is there a small canonical basis of recursion schemes with measured coverage? . . .	18
9	R6: Contracts beyond finite decidable observations	19
9.1	R6.Q1. How should value construction and proof obligations be interleaved?	19

9.2	R6.Q2. Can one recognize properties provable by program-aligned induction and arithmetic?	20
9.3	R6.Q3. What induction automation is usable in Lean now?	20
9.4	R6.Q4. Is there a principled scheduler for proof debt?	21
10	R7: Component retrieval at library scale	22
10.1	R7.Q1. What abstraction is useful for dependent, universe-polymorphic signatures?	22
10.2	R7.Q2. Has type-guided abstraction refinement been combined with type classes?	22
10.3	R7.Q3. Can existing premise retrieval support data goals and examples, and where should training data come from?	23
11	R8: Ranking and candidate equivalence	24
11.1	R8.Q1. What formal objective is defensible for ranking inhabitants?	24
11.2	R8.Q2. Which candidate equivalence is cheap and meaningful for dependent types?	24
12	R9: Learned guidance under a kernel gate	25
12.1	R9.Q1. What accuracy do current models achieve on accumulator or helper proposals?	25
12.2	R9.Q2. Can learned contributions be deterministic and auditable beyond recording proposals?	26
13	An integrated implementation plan	26
13.1	Four interfaces with different authority	26
13.2	State ownership and a resumable transition system	27
13.3	Recommended order of implementation	28
14	Evaluation, falsification, and reproducibility	29
14.1	A benchmark design that diagnoses causes	29
14.2	Adversarial tests for the claimed guarantees	30
14.3	Checks actually executed for this article	30
15	Conclusions	31
A	Question-by-question coverage	31
B	Source audit and scope of verification	33
C	Using the accompanying package	34
	References	35

1 Scope, evidence, and the main recommendation

The target is not a generic theorem prover. It is a Lean-native synthesizer that accepts a type and an optional behavioral contract, produces inhabitants, and checks the resulting program and evidence against a trust profile. That purpose makes the interaction between data construction, proof search, and evaluation more important than any one technique in isolation.

The repository was inspected at commit

784664ff34e819422510a4d470ab07fe48bb736b.

The main evidence is the complete set of research sections in docs/proposals/11-further-improvements, the implementation notes, and selected source modules for construction search, acceptance, and transactional state. The proposal’s benchmark and profile figures refer to an earlier measured commit, 33cec8b, using Lean 4.34.0 on one Windows workstation. Those figures are treated as reported results, not as measurements repeated for this article [1, 2, 3].

The proposal reports complete outcome-category agreement on its scored Leant suites, while identifying thirteen unscored Church-operation stretch cases and additional failures just outside the corpus. That is evidence that the existing search is useful and that the saturated baseline is insufficient for choosing future algorithms. It is not evidence that all those failures have the same cause or that a rewrite will improve them. Likewise, the reported 45% residual-evaluation cost and 1% snapshot cost come from a particular contract-heavy profile, not a universal cost model [2].

The central recommendation is to retain the current kernel-based acceptance boundary while making three distinctions architectural rather than informal: *what may be proposed*, *what may be pruned*, and *what may be claimed*. An arbitrary carrier guess can be useful. An arbitrary carrier refutation can silently remove the only solution. A checked program is useful without being the smallest program, the intended program, or a proof of nonexistence of alternatives. A stronger engine needs to represent these differences explicitly.

1.1 Kinds of conclusion used in this article

A *published result* is attributed to a primary paper or an official implementation. A *derived claim* is accompanied by an argument and its assumptions. An *engineering recommendation* is a proposed change, not a measured improvement. A *bounded guarantee* names the language, resources, and decision procedures within which it holds. An unanswered empirical question is accompanied by an experiment rather than a guessed number.

The mathematical propositions below are ordinary written proofs, not Lean formalizations. The accompanying Python program checks finite examples and small exhaustive models; it neither executes Leant2 nor validates Lean’s kernel. The distinction matters especially for higher-order unification, partial evaluation, and performance: the proposed adaptations have not been implemented or benchmarked here.

2 Corrections that change the plan

Higher-order examples are not an unfilled Smyth feature. The Smyth paper explicitly distinguishes a first-order presentation of its core calculus from an implementation supporting higher-order function examples. It also removes the requirement that users supply trace-complete examples. Thus the proposal’s statements that Smyth’s unevaluation is simply first-order and that it shares the trace-completeness requirement should be revised. Full dependent-type support remains a separate adaptation [15].

Auxiliary-state invention has substantial precedents. AutoLifter synthesizes auxiliary computations and combinators for algorithmic lifting; Eguchi, Kobayashi, and Tsukada synthesize auxiliary function types in templates; Para uses paramorphism-based synthesis templates. None is a drop-in solution to the exact Lean problem, but together they invalidate a broad conclusion that the literature does not address auxiliary state or types [17, 18, 19].

Type classes already occur in type-guided abstraction refinement. Hoogse+’s Section 2.5 explicitly uses dictionary passing. The unresolved part is the interaction with Lean’s dependent signatures, universes, local instances, and computational dictionaries, not the basic possibility of representing class constraints in a synthesis net [16].

Indexed induction is partly an integration problem. Canonical explicitly supports indexed inductive types and generalized induction. In Lean 4.34.0, `Lean.Meta.mkRecursorAppPrefix` constructs a motive by abstraction, and `MVarId.induction` already reverts indices and the major premise with their dependencies. Leant2 must still select useful extra generalizations, but it should not reimplement this existing machinery as though it were absent [14, 7].

Several proposed guarantees need stronger hypotheses. One unit per unfinished goal can overestimate remaining search cost. Different partial observation rows need not describe different states. Values already enumerated for a carrier are not an upper bound on its possible values. Ranking after parallel lanes stop does not restore candidates that a slower lane never generated. These are mathematical issues, not tuning questions; counterexamples and corrected guarantees are given below.

Two concrete examples should be replaced. Ordinary list reversal is a right fold into *the result type*:

$$\text{reverse}(xs) = \text{foldr} (\lambda a r. r ++ [a]) [] xs.$$

The issue is the availability and cost of the step, not the nonexistence of such a carrier. Also, for Lean’s usual natural-number addition, $n + 0$ reduces definitionally to n ; use $0 + n$ with symbolic n , or a commuted addition, to illustrate a genuine propositional index transport. These observations are used in the carrier and transport designs, respectively [9].

3 The semantic contract of the search

Fix an imported environment E , a local telescope Γ , a target type T , and a contract $C : T \rightarrow \text{Prop}$. Open terms are interpreted with a metavariable state σ , including universe assignments and delayed constraints. Write $\text{Comp}_B(p, \sigma)$ for the well-typed completions admitted by an explicitly specified bounded grammar B . Omitting B means all admissible completions under the chosen language and trust policy.

Definition 3.1 (Three independent obligations). *Acceptance soundness* requires an accepted t and π to check as $t : T$ and $\pi : C(t)$ in the intended environment. *Conservative pruning* requires

$$\text{reject}(p, \sigma) \implies \forall t \in \text{Comp}(p, \sigma), \neg C(t). \quad (1)$$

Optimality requires that the returned answer minimize a specified objective over a specified candidate set. Neither of the first two obligations implies the third.

A bounded rejection may replace Comp by Comp_B , but its receipt must then name B . A failed tactic invocation does not establish (1). A timeout does not establish bounded exhaustion. A passing test establishes that test, not an untested universal contract. These distinctions should survive all the way to the result algebra and user interface.

The inspected gate synchronously checks declarations, generalizes universes jointly over the program and proof, and audits their transitive axiom closure. Its proof type is supplied by its caller. This is a reasonable internal API under a trusted caller invariant, but an externally exposed synthesis service should take the original query and reconstruct $C(t)$ inside the gate. Checking an arbitrary supplied proof type would not by itself establish the requested contract. This is an API-hardening recommendation, not a demonstrated end-to-end defect in the current callers [6].

Three result forms should therefore be distinct: a certified solution; a checked impossibility proof or a genuinely exhausted, named finite search; and an incomplete search with unrefuted survivors. Evidence supporting a finite contract is still a fully valid proof of *that finite contract*. It simply does not identify the unstated intended algorithm.

4 R1: Search control with coupled goals

4.1 R1.Q1. Can shared-metavariable goals be organized without wholesale copying or sequentialization?

Assessment: Yes, through conditional factorization and tabling; no, if the demand is to erase the information distinguishing incompatible assignments.

Represent a node as a constraint hypergraph. Its vertices include unfinished term holes, universe variables, postponed constraints, and observations. A hyperedge records a joint dependency. Dependencies must be closed through assigned expressions, delayed assignments, local declaration types and values, and instance arguments. For example, two goals whose printed targets differ may depend on the same universe variable or on a local whose type contains a shared hole. Direct comparison of target metavariable sets misses this.

A component can be solved independently only after its external interface is fixed, or while its solutions remain parameterized by that interface. Store a solution as an abstracted term and a constraint on its interface, not as an unqualified assignment to a global metavariable identifier. On reuse, instantiate fresh variables, check the interface constraints, and replay the term through type checking. For small separators this resembles variable elimination: each side exports a relation on a few shared variables rather than every internal branch.

Proposition 4.1 (A sufficient factorization criterion). *Suppose a state's completion constraints are $D_1 \wedge D_2$, where the unassigned variables of D_1 and D_2 are disjoint after dependency closure. Assume a fixed common telescope and that every allowed action on either side preserves this separation. Then compatible solutions of D_1 and D_2 combine by disjoint substitution, and every joint solution restricts to a solution of each side.*

Proof. The substitutions have disjoint domains and no right-hand-side dependence on the other side's unassigned variables. Their union is therefore a well-defined, capture-avoiding substitution. Substitution into D_i is unaffected by the other component, so both constraints hold. Restriction gives the converse. The action-preservation condition prevents later creation of a hidden shared constraint from invalidating the factorization. $\square$

This is a sufficient condition, not a cheap complete independence test. Start with syntactic dependency closure and conservatively retain uncertain edges. Recompute only after relevant assignments or new

observations. Cache successful, abstracted solutions first; caching failure requires a certificate of exhaustion at a particular grammar and resource bound.

Persistent contexts already share most unchanged structure. Aesop already computes transitive clusters of coupled goals, including dependencies through local contexts. Its copying concerns goal nodes, not wholesale copies of their search subtrees. Thus cheap independence detection is not a missing idea in Aesop; the additional proposal here is interface-parameterized solution tabling and carefully limited factorization [13]. Incompatible assignments still require distinct logical branches.

There is also an immediate implementation caveat: `Meta.SavedState` is not the whole current search state. The residual-blocker array and several other fields are `IO.Refs` outside the snapshot. A queued node needs its own residual-cache identity and resumable expansion cursor. Global work charges should remain outside snapshots, but branch-dependent semantic caches must not be shared without valid keys [4, 5].

4.2 R1.Q2. What admissible heuristic remains valid when unification closes several goals?

Assessment: Use zero as the baseline, maximum rather than sum for coupled goals, and additive bounds only across action-separable components.

Suppose two unfinished obligations are a data hole $x : \mathbb{N}$ and an equality hole $p : x = 7$. An equality-solving step may instantiate x while solving p . If that step costs one, charging one for each obligation gives an estimate of two for a remaining cost of one. Even an isolated goal may have a zero-cost exact solution under the proposal’s weights. The number of visible goals is therefore not automatically a lower bound.

Proposition 4.2 (Combining lower bounds). *Let $c^*(s)$ be the minimum additional cost of completing state s , with nonnegative action costs. If $h_i(s) \leq c^*(s)$ for every i , then $\max_i h_i(s) \leq c^*(s)$. A sum of component bounds is admissible if every completion’s cost can be partitioned into component costs, and the bound for each component is no greater than its corresponding share.*

Proof. The maximum cannot exceed a common upper bound. Under the partition hypothesis, sum the inequalities for the component shares. Without that hypothesis, one action can pay for several components and the sum double-counts. $\square$

A better nontrivial bound comes from a finite abstraction of the remaining constraint system. Give every concrete action an abstract counterpart of no greater cost; allow the abstraction to forget constraints and to close several abstract goals together. Abstract shortest-path distance is then admissible: every concrete completion projects to an abstract completion of no larger cost. An abstraction that merely treats all goals independently does not have this property without an additional proof.

Use a learned estimate of difficulty for ordering, but do not call it an admissible heuristic. Keep an immutable logical cost for optimality claims. Also, zero-cost rules need either an acyclic normalization phase, duplicate suppression with a termination argument, or a structural bound: infinitely many zero-cost expansions can otherwise prevent progress in cost order.

4.3 R1.Q3. Can just-in-time learning preserve meaningful ranking?

Assessment: It preserves logical soundness under the gate, but neither budget invariance nor an immutable ranking objective follows automatically.

Probe is an important precedent for using partial success to guide enumeration, not a theorem that learned search order is independent of the examples, execution trace, or stopping time [20]. Maintain two quantities:

$$c_0(t) = \text{fixed description cost}, \quad q_e(s) = \text{exploration priority during epoch } e.$$

Use q_e to allocate search effort and c_0 to compare completed candidates. A learning update must occur after a deterministic logical event, such as a fixed number of completed expansions, not after a wall-clock interval or a race between parallel workers. Record the update and its inputs.

For best-first search with changing priorities, either rebuild affected queue keys or freeze priorities for a bounded epoch. An A* proof about old keys is not valid after arbitrary weight changes. A fixed-cost companion frontier can supply a lower-bound certificate even while a heuristic lane explores other nodes. A reserved fair share for that companion prevents learned priorities from starving a valid bounded alternative.

Users should see “best found under this search receipt,” unless the fixed frontier proves a stronger statement. A reference solution moving in the list because the algorithm found better evidence is defensible; pretending that an adaptive search returns a machine-independent optimum within ten seconds is not. Learning may use the proportion of observations satisfied, but the observations should be weighted carefully: forty correlated tests should not necessarily overwhelm one structurally different test.

4.4 R1.Q4. Can a hard wall-clock budget give machine-independent results?

Assessment: Not in general. Deterministic prefixes, replay, and completed logical epochs are achievable alternatives.

Proposition 4.3 (A wall-clock obstruction). *Consider a deterministic algorithm whose answers change after some positive amount of computation. If execution speed may vary sufficiently, a common hard wall-clock cutoff cannot guarantee the same answer set on every execution while also publishing all answers reached before that cutoff.*

Proof. Choose a logical event at which the answer set changes. Choose two speeds so that the event occurs before the deadline on one execution and after it on the other. Publishing the answers reached gives different sets. Suppressing the new answer avoids that pair of outputs only by abandoning the requirement to publish all reached answers. $\square$

Even a restricted speed variation breaks equality whenever a cutoff straddles such an event. A known worst-case execution bound can support a fixed amount of logical work, but then the guarantee is conditional on that bound. Startup calibration is an estimate, not a proof; cache warmth and input-dependent kernel costs can invalidate it.

The practical contract should be *prefix reproducibility*: pin the query, environment, grammar, ranking model, and tie-breaking; number logical expansions; publish a resumable trace prefix. A deterministic-work run can reproduce that prefix regardless of elapsed time. A timed run returns the prefix it managed to finish. Completed epochs may be compared across machines, but their number need not match. Parallel workers may explore speculatively, while a deterministic merger commits results in logical order.

Luby-style restart schedules give useful guidance under their Las Vegas runtime assumptions; arbitrary coupled, stateful lanes do not automatically satisfy those assumptions [21]. Prefer resumable lanes and deterministic work allocation before adding restarts. Replaying the same initial search because a deadline interrupted it is wasted work unless the restart changes something that justifies its cost.

5 R2: Carriers, accumulators, and auxiliary state

This is the most valuable place to combine a precise mathematical model with practical synthesis templates. A fold carrier is not just a type: it is a representation equipped with an initializer, a transition, and an observation map. Searching the type alone discards most of the useful information.

5.1 R2.Q1. Can finite examples determine a minimal carrier type, or prove that none exists?

Assessment: A bounded constructive problem is solvable; the unrestricted request confuses semantic minimal state with a minimal representation in a type grammar.

For an ordinary list function $h : A^* \rightarrow B$, consider a realization

$$q([]) = z, \quad q(a :: xs) = s(a, q(xs)), \quad h(xs) = o(q(xs)), \quad (2)$$

with carrier K , initializer $z : K$, step $s : A \times K \rightarrow K$, and observer $o : K \rightarrow B$. Define a residual equivalence on lists by

$$x \sim_h y \iff \forall u \in A^*, h(u ++ x) = h(u ++ y). \quad (3)$$

The orientation is important: a right fold extends an already-processed tail by prepending more elements.

Theorem 5.1 (The semantic carrier). *If q realizes h as in (2), then $q(x) = q(y)$ implies $x \sim_h y$. The quotient $A^*/\sim_h$ itself supports a realization of h . Consequently, every reachable realization has at least as many states as this quotient, in the sense that its reachable states map surjectively onto the quotient.*

Proof. If $q(x) = q(y)$, induction on the prepended list u gives $q(u ++ x) = q(u ++ y)$; applying o proves the first claim. For the quotient, take $z = [[]]$, $s(a, [x]) = [a :: x]$, and $o([x]) = h(x)$. The step is well-defined because every test of $a :: x$ against $a :: y$ is a test of x against y with context $u ++ [a]$. The observer is well-defined by taking $u = []$. Finally, map a reachable state $q(x)$ to $[x]$. The first claim makes this well-defined, and every class is reached. $\square$

This is the residual-state viewpoint behind fold characterizations and automata minimization, specialized to the proposal's question; it is not a new universal carrier-inference algorithm [22]. The quotient may be infinite, its equivalence may be undecidable, and it need not have a usable representation in the proposed Lean type grammar. Even a finite semantic carrier has many representationally equivalent types. Minimal cardinality, minimal type-expression size, minimal program size, and minimum runtime are different objectives.

Finite samples do not identify (3). On an infinite domain, two computable functions can agree on every supplied example but disagree elsewhere. Without restricting the step and observer, storing the entire input list and applying h at the end always gives a semantic realization. Thus an existence claim without a language or cost restriction can be almost vacuous, while a claim of the intended minimal carrier can be unidentifiable.

A useful decidable version fixes a finite alphabet, a finite carrier of k states, a finite output alphabet, and finitely many observations. Enumerate transition and observation tables, or encode them as a finite constraint problem. For a two-symbol alphabet and Boolean outputs, fixing the initial state to 0 leaves $k^{2k} 2^k$ tables. Searching $k = 1, 2, \dots$ gives a smallest *sample-consistent finite-state machine*. This is constructive, but not an identification theorem for the intended infinite function.

For the Lean grammar, use bounded type expressions and bounded typed programs for z, s, o , with finite literal and component frontiers. Include proof and universe bounds when they are searched. Exhaustive enumeration is a decision procedure only when every membership and observation check in that finite fragment terminates decisively. A heartbeat timeout must leave the result unknown, not become a proof that no representation exists.

A sound early refuter. Build a graph on observed tails. Connect x and y when some *common* context u has known unequal outputs for $u ++ x$ and $u ++ y$. Every realization assigns different states to adjacent vertices. A clique of size $k + 1$ therefore refutes a carrier with at most k possible reachable states. More strongly, the chromatic number is a lower bound, although transition congruence can impose further constraints.

Distinct incomplete rows are not enough: a row containing only $c_0 \mapsto \text{false}$ and a row containing only $c_1 \mapsto \text{true}$ are different but compatible. Nor is the set of values already enumerated for K an upper bound on all possible states. Repeated application of a tiny transition can build states absent from that enumeration. Use the actual finite cardinality or a proved reachable-state bound.

Example 5.2 (Boolean output, three necessary states). Let $h(xs)$ mean that the length of xs is divisible by three. Tails of lengths 0, 1, 2, tested with contexts of lengths 0, 1, 2, give the rows

$$(T, F, F), \quad (F, F, T), \quad (F, T, F).$$

They form a three-clique. A Boolean carrier is impossible even though the output is Boolean. A three-state carrier exists: increment the length class cyclically at every step and observe whether it is zero. Its correctness follows by induction on the list. The finite-model suite additionally exhausts all one- and two-state machines against 31 binary-list samples of length at most four; none fits. A three-state witness is found.

5.2 R2.Q2. What replaces an alphabet for polymorphic functions?

Assessment: Positions are justified in an audited parametric fragment; equality patterns require an exposed equality operation and a different relational model.

Suppose a family $h_A : \text{List}A \rightarrow \text{List}A$ is natural: it commutes with `map` for every function between element sets. For an input $xs = [x_0, \dots, x_{n-1}]$, let $e_n = [0, \dots, n-1]$ over $\text{Fin}(n)$ and let $v : \text{Fin}(n) \rightarrow A$ select the corresponding element of xs . Then naturality gives

$$h_A(xs) = \text{map}(v)(h_{\text{Fin}(n)}(e_n)). \quad (4)$$

Thus the output is described by a word of input positions. The same equation handles repeated elements, because v need not be injective. The empty input is covered using the empty finite type. This is the precise reason canonical position-valued tests are valuable, rather than an appeal to polymorphism as an informal slogan [23].

Equation (4) is conditional on a naturality theorem. The surface Lean type alone is not a certificate that arbitrary imported providers, classical operations, or dictionaries satisfy it. Establish a syntactic parametric fragment or require relational preservation theorems for admitted components. A raw `BEq A` dictionary in particular is not automatically lawful or parametric in this sense.

When elements are abstract atoms equipped with equality, the relevant symmetry is renaming that preserves the available operations. Equality patterns and a bounded set of stored atoms can then be represented by nominal or register-style state spaces. Nominal automata supply a mathematical framework for structured infinite alphabets [24]. This does not make every list transformation finite-state: copying or reversing arbitrary lists can require unbounded stored data. Separate a finite control state from symbolic payloads such as saved elements or lists.

Testing canonical inputs up to length N can establish equality on lists of length at most N under the naturality hypothesis. It does not establish global equality or equality of all programs of syntactic size at most d unless a separate small-model theorem relates d to N .

5.3 R2.Q3. Can fusion run backward to invent auxiliary information?

Assessment: Yes as constrained lifting and feature discovery; no general complete inference of an intended helper follows from a few examples.

AutoLifter is particularly relevant: it decomposes lifting problems into auxiliary computations and combiners, using a supplied original computation as a source of observations. Its approximation choices can generate unrealizable subtasks. Therefore it is neither an unrestricted minimal-type procedure nor a method requiring only a fixed handful of examples [17]. Eguchi and colleagues supply another concrete precedent for inferring auxiliary function types in templates [18]. These are adaptation starting points rather than reasons to reinvent the entire problem.

For Leant2, represent a proposed carrier together with a symbolic invariant $J(xs, k)$ explaining the information stored in k . Search the three local obligations

$$J([], z), \quad J(xs, k) \implies J(a :: xs, s(a, k)), \quad J(xs, k) \implies C_x(o(k)). \quad (5)$$

Here C_x denotes the required result property at input xs , with parameters made explicit. For a reference implementation it can be equality to the reference; for a finite contract it is only an observation constraint. The invariant is a proposal until its obligations are proved.

Use failed symbolic closure to invent features. If the next output requires an old quantity not reconstructible from the current summary, add that quantity to the state. For Fibonacci, a proposed state F_n is insufficient to update uniformly by the elementary recurrence without further information. The pair (F_n, F_{n+1}) admits the closed transition $(a, b) \mapsto (b, a + b)$. The initializer $(0, 1)$ and induction immediately justify the invariant. This is a useful synthesis template, not a proof that no clever single-natural encoding exists.

For reversal, the efficient endomorphism carrier has

$$K = \text{ListA} \rightarrow \text{ListA}, \quad z = \text{id}, \quad s(a, k)(acc) = k(a :: acc), \quad o(k) = k([]).$$

An induction gives $q(xs)(acc) = \text{reverse}(xs) ++ acc$. The ordinary list-valued fold also exists but repeatedly appends at the end. For minimum and maximum together, an optional pair records both the empty case and two extremes. A total maximum or index operation must make its empty or out-of-range behavior explicit; inventing a carrier cannot create an arbitrary element of an uninhabited type.

A complete useful class is obtained by fixing a finite feature vocabulary, a finite candidate transition language, and decidable closure obligations. Enumerating feature subsets and their bounded transitions is then complete for that class. It is an intentionally limited synthesis problem, not a complete strategy for all catamorphisms over all polynomial functors.

5.4 R2.Q4. How can motives for primitive recursion with parameters be enumerated?

Assessment: Enumerate a typed, bounded family of motive templates; completeness is relative to that family and its bound.

Separate parameters fixed throughout recursion from changing accumulators. Given a recursor argument $x : I\vec{t}$ and a result family T , begin with its goal-directed motive. Extend it in increasing structural cost by a function space for selected changing parameters, a product for extra outputs, and an optional state for initialization. Each extension includes its initializer, observation map, and invariant obligations. This couples motive selection to the intended computation instead of enumerating arbitrary functions into a universe.

A useful template has the form

$$M(\vec{t}, x) = \Pi(a_1 : S_1) \cdots (a_m : S_m), K(\vec{t}, x, \vec{a}),$$

where K comes from a finite carrier grammar and each binder telescope is well-formed in dependency order. At a fixed size bound there are finitely many templates if types, literals, and component choices are also bounded. Enumerate them once, reject ill-typed ones early, and share residual constraints between compatible

templates. Increasing the bound fairly gives coverage of the grammar’s union, not a terminating procedure for all Lean inhabitants.

For natural-number recursion, try a single outer recursion capability before allowing nested recursor use. The proposal attributes an earlier explosion to nested induction on hypotheses. That is a reason to control capability scope and template nesting, not to exclude `Nat` from the language permanently [3]. Para provides relevant empirical experience with recursion templates, including function-valued states [19].

5.5 R2.Q5. Do induction-generalization heuristics transfer to carrier selection?

Assessment: Yes as proposal generators, especially when guided by a specific failed step obligation.

Record why a recursive step failed. If the induction hypothesis is about a fixed accumulator while the step changes it, propose generalizing that accumulator. If the step needs two related outputs of the recursive call, propose tupling. If a symbolic output comparison leaves a recurring expression unmatched, propose that expression as a feature of the invariant. If a common subterm appears across failed obligations, propose a helper with that subterm’s type and an example relation.

This is the useful transfer from rippling and lemma speculation: compare the induction hypothesis with the desired conclusion and isolate a structural obstruction. It should not be implemented as “a failed proof proves that this carrier is wrong.” Failure may instead reflect insufficient simplification, a missing library theorem, or inadequate resources. Modern induction work also investigates generalization inside saturation provers; its search principles are relevant even when its proof format is not directly reusable in Lean [29].

Use a deterministic, small menu of obstruction-to-template transformations. Charge every proposed helper and generalized binder in the grammar budget, and retain the nongeneralized branch. A successful synthesized invariant is then proved by the local obligations in (5). The highest-priority experiment is a three-track benchmark: carrier supplied, carrier selected from a fixed template list, and carrier inferred from failure evidence. This separates carrier discovery from the cost of synthesizing its transition.

6 R3: Evaluation of programs with holes

6.1 R3.Q1. What is already available for higher-order and dependent examples?

Assessment: Higher-order examples have a concrete precedent in Smyth; dependent closures need additional typed-substitution machinery.

Smyth represents incomplete computations by hole closures and propagates required results backward into constraints on holes. Its implementation supports higher-order function examples even though its core exposition is simplified to first-order examples [15]. Leant2 should begin by adapting that distinction: higher-order behavior is observed through a finite collection of applications, not by demanding a complete mathematical function as the output of unevaluation.

For a hole h declared under telescope Δ , represent an observation as

$$(h, \rho, O), \quad \rho : \Gamma \longrightarrow \Delta, \tag{6}$$

where ρ is a well-typed environment substitution and O is a typed elimination or result constraint. Two occurrences of h can have substitutions that instantiate indices differently. A hole producing `Vec B n` may be observed at length zero in one closure and length three in another. These are constraints on one contextual term, not two untyped examples of a common ground output type.

The first dependent difficulty is therefore not merely storing proofs in an environment. It is transporting the expected observation type along ρ and composing substitutions correctly under binders. The next difficulty is backward propagation through an eliminator whose motive depends on the value being inspected. Begin with a typed first-order data fragment, then dependent constructors with known indices, then application observations of function holes. Unsupported inversions should produce residual constraints rather than inventing an untyped example.

A contract is not necessarily a list of equations. Normalize its Boolean and propositional structure using checked equivalences, retaining provenance to the original proposition. Splitting `List.all` over a known finite list or a conjunction of Booleans is useful. Replacing `a == b = true` by `a = b` needs a suitable lawfulness theorem; `BEq` alone does not supply one. Multiple calls to the synthesized function in one observation must remain related rather than being treated as independent input-output examples.

No source examined here supplies a ready-made, conservative evaluator for the whole Lean language with this dependent closure interface. That identifies an implementation and proof task, not evidence that every component of it is unexplored.

6.2 R3.Q2. How should normalization caches survive assignment and rollback?

Assessment: Use dependency-tracked persistent memoization; do not key validity only by hole identifiers or a global update counter.

Self-adjusting computation provides a general precedent for memoization with change propagation and a consistency theorem [25]. It is not a drop-in normalizer for Lean. The adaptation is to record what an evaluation *read*, not only what variable it was visibly stuck on.

A memo entry should contain the expression or closure identity, local-context fingerprint, relevant environment and transparency data, a result, and a read set. Each read-set item records an assignment's identity or immutable version. Include universe assignments, delayed-assignment links, instance dictionaries, and type dependencies whenever the evaluator inspected them. Reuse is valid only when all relevant reads still denote the same values.

Proposition 6.1 (Read-set cache reuse). *Assume a deterministic evaluator's result depends only on an immutable expression, fixed semantic parameters, and the values in its recorded read set. If another state agrees on all those values, the memoized result is valid there.*

Proof. Replay the evaluation. Every read yields the recorded value, so every branch and reduction rule is the same. Induction over the finite evaluation trace gives the same result. The completeness of the read set is an explicit hypothesis, not a property implied by recording just blocked holes. $\square$

A rollback can restore an old assignment and later take a different branch using the same metavariable identifier. This is the familiar identity-reuse problem: the name did not change, but its meaning did. Immutable state tokens, content identities, or validated assignment fingerprints avoid it. A single global timestamp is safe if it invalidates everything on every relevant change, but loses most reuse; a reused timestamp is unsafe.

Use per-observation caches first. The current code already decomposes conjunctions and remembers blockers, so this is an incremental improvement. However, `residualBlockers` is outside the saved meta-state. A future frontier must attach its memo identity to the node, or validate the entire read set before reuse [4, 5]. False results can be cached globally only with a proof or a key strong enough to establish that the exact same proposition and program are being refuted.

6.3 R3.Q3. What is the cheapest route to a provably conservative fast evaluator?

Assessment: A small proof-producing or reflected fragment is preferable to a second unverified implementation of the entire kernel.

The evaluator need not decide every Lean term. It needs to prove the decisions it does make. Define a typed observation language containing the constructors, projections, primitive operations, and recursors the synthesizer actually uses. Give it a denotation into Lean terms and prove a fuelled evaluation correctness theorem. A successful evaluation can then provide a theorem about the original observation; unsupported syntax or fuel exhaustion gives *unknown*.

There are two practical implementation paths. A proof-producing reducer emits an equality certificate for each step, composing certificates only for the parts inspected. A reflected evaluator runs over a reified typed syntax and uses one proved soundness theorem. The former has simpler incremental integration; the latter may give smaller checking overhead after a larger initial proof investment. Both permit a fast untrusted proposal phase followed by validation of the claimed reduction.

Proposition 6.2 (Residual refutation). *Suppose a partial program p has a well-typed completion telescope Δ . If a checked certificate proves*

$$\forall \theta : \Delta, \quad \text{decide}(C(p[\theta])) = \text{false},$$

using an appropriate decision procedure for each instance, then rejecting p is conservative. A certificate under additional assumptions is sufficient only when those assumptions follow from the requested contract for every accepted completion.

Proof. For any completion, instantiate the certificate. The correctness of `decide` yields $\neg C(p[\theta])$. Under additional assumptions, a hypothetical accepted completion supplies those assumptions and produces the same contradiction. $\square$

The current kernel-reduction path has a strong advantage: genuine reduction of a well-typed expression commutes with well-typed substitution. But its `instantiatePartial` helper explicitly constructs reduction-only expressions whose metavariable contexts may no longer match. A completeness argument still needs a well-scoped closure/substitution account for that transformation. Calling the kernel's reducer does not by itself prove a separate preprocessing transformation correct [4].

Lean4Lean is a useful source of formalized kernel structure and verification work, not a button that extracts an incremental closure evaluator [26]. A closure machine has its own environment relation and substitution lemmas. Reuse relevant definitions and proved lemmas where they match, but prove the fragment's simulation theorem explicitly. Differential testing remains valuable for finding mistakes; even a hundred thousand passing closed tests is not a proof about every open completion.

6.4 R3.Q4. Can top-down angelic recursion have termination and completeness guarantees?

Assessment: Yes for carefully bounded fragments with fair exploration and relational oracle constraints; not by simply guessing one helpful return value per call.

An angelic recursive call denotes an existentially chosen result subject to constraints. Equal arguments to the same pure function must share a result. For example, independently choosing two results could satisfy $f(0) \neq f(0)$, although no function can. Maintain a map of call observations with equality and relational constraints, rather than unrelated successful oracle guesses.

A supplied equation $f(v) = w$ can constrain the oracle at v because every accepted function must respect it. An unobserved recursive result is different: a convenient value may guide a branch, but failure of that

value does not refute all possible results. To prune using angelic execution, compute an over-approximation of possible results and prove it cannot satisfy the observation. Otherwise use angelic information only for ordering.

Burst gives a concrete refinement-based synthesis design, but its arguments cannot be transferred merely by changing bottom-up enumeration to top-down search [27]. A top-down bounded theorem needs finite skeletons, finite oracle domains or a terminating complete constraint solver, fair exploration of refinements, and a decreasing or bounded measure preventing repeated unchanged refinements. Under those conditions the search terminates; if its rules cover every bounded candidate and all pruning is conservative, it is complete for that fragment. For unbounded syntax, dovetailing yields at most a relative semidecision guarantee. No general termination claim follows from the existence of a final kernel gate.

7 R4: Indexed families and motive search

7.1 R4.Q1. Can a useful motive class be enumerated completely and terminate?

Assessment: Yes at a fixed template bound; index-occurrence abstraction alone is not closed under accumulator generalization.

A practical motive search should start from the recursor’s dependent telescope, not from an untyped enumeration of expressions of type `Sort`. Compute candidate abstractions of selected, maximal occurrences of indices and the major premise in the goal. Abstract consistently, preserve dependencies, and reject any candidate whose motive type is wrong. If there are k eligible occurrences, a raw subset enumeration is finite, with at most 2^k choices; typed equivalence and repeated occurrences can reduce it substantially.

That finite class does not automatically include accumulating recursion. A motive like $M(n, x) = S \rightarrow T(n, x)$ contains a new binder not created by replacing an index occurrence. Close the template language explicitly under a bounded number of dependent products, product-valued strengthening, and optional initialization state. Enumerate dependency-closed subsets of existing locals for generalization and bounded fresh auxiliary types from the carrier grammar. This gives an exact, useful answer to the question: completeness relative to the defined template language, not completeness for arbitrary motives.

For $\text{Vec } A \ n \rightarrow \text{Vec } B \ n$, the intended motive abstracts the index and vector to yield $\text{Vec } B \ n$; no accumulator invention is necessary. Preserve fixed parameters A, B , and the mapping function, and give the recursive branch its correctly indexed induction hypothesis. Generalizing every local would unnecessarily enlarge the branch interface. Conversely, an accumulator that changes at recursive calls must be generalized even when its type does not depend on an index. Syntactic index-dependency closure alone will miss it.

7.2 R4.Q2. How should transports be represented without destroying canonical enumeration?

Assessment: Canonicalize index expressions where justified, retain typed equality evidence, and separate display erasure from kernel terms.

Proof irrelevance concerns inhabitants of the same proposition. It does not turn every pair of propositionally equal indices into definitionally equal indices, nor does it make every `Eq.rec` computation disappear. A term of $\text{Vec } A \ n$ cannot simply be inserted where $\text{Vec } A \ m$ is expected because some proof of $n = m$ exists somewhere. The equality and the appropriate transport must actually be supplied.

Use a small index-normalization service. It returns a normalized index and a proof of equality to the original, or reports that no supported normalization applies. Constructor arithmetic and selected associativity/identity rules can be handled predictably. Arbitrary `simp` search can be an optional proof service, but its success is not

a definition of a globally canonical form. The example $\text{Vec } A \ (n + 0)$ needs no propositional cast with Lean’s usual addition; $\text{Vec } A \ (0 + n)$ at symbolic n is the instructive case [9].

A typed congruence structure may store equivalent expressions together with proofs and their types. Heterogeneous equality can help express relationships before source and destination types coincide, but it does not remove the need for a typed reconstruction theorem. Replacing Lean’s typing problem by a setoid problem would move rather than solve the difficulty.

Rank candidates using a proof-insensitive representation only after stating which equalities it respects. Proof subterms may have zero semantic ranking cost, while transport nodes still have construction and checking costs. A displayed term with omitted casts must be elaborated again and proved equal to the certified term, or be explicitly labeled as a presentation rather than paste-ready Lean. Bound transports to control search, but record that bound; it restricts the grammar and may exclude valid programs.

7.3 R4.Q3. What unfolding strategy is supported by existing experience?

Assessment: Use staged, demand-driven unfolding in Leant2, informed by actual tool behavior rather than a universal transparency rule.

Canonical’s published encoding unfolds definitions that alias function types, including `Set` and `Not`, and unfolds reducible definitions and type-class instances before constructing its search problem. This is more specific than describing it simply as fixed transparency or complete normalization. It also supports generalized induction over indexed types, so it is a stronger baseline than the proposal’s comparison suggests [14].

The sensible adaptation is a staged policy. First expose only the head shape needed by the next rule: a function binder for introduction, an inductive head for elimination, or a class head for instance resolution. Use a small, predictable reducible view for indexing. Escalate transparency only when a particular matching or conversion attempt is stuck, and charge the escalation to that attempt. Cache under the transparency mode and relevant environment fingerprint. Do not normalize every program and every local type fully at every node.

The official sauto documentation gives a concrete comparison: `unfold`: names definitions to unfold heuristically, with an empty list by default; `unfold!`: forces the listed unfoldings. Logical iff and not receive special treatment, while red simplification and its eager scheduling are enabled by default. This is a configurable mixture of reduction and selective unfolding, not blanket normalization of all definitions [30]. Pin the version and these options in any comparison; the documented defaults are not a theorem about every historical implementation path.

A useful ablation measures head exposure, reducible normalization, and aggressive unfolding separately. Count conversion failures, expansions, normalization time, and cache growth. A transparency policy that saves a few unifications but expands huge definitions is not necessarily beneficial.

7.4 R4.Q4. Can Lean’s motive computation be called directly with useful failures?

Assessment: Yes. Lean 4.34.0 exposes concrete meta-level entry points; their cost and diagnostic abstraction must be controlled by an adapter.

Three operations in the inspected source are directly relevant. The function `getMajorTypeIndices` extracts the recursor’s indices. The function `mkRecursorAppPrefix` constructs a recursor prefix through its motive by abstracting the major premise and indices from the target. Finally, the goal method `MVarId.induction` already reverts indices and the major premise, reintroduces them, builds the prefix, and returns branch goals with a free-variable substitution. These operations are not all hidden inside syntax elaboration [7].

The low-level index extractor requires variable indices satisfying specific occurrence and dependency conditions. A major premise with a compound or repeated index may therefore need the higher-level

preparation used by cases/induction, or an explicit generalization with equality constraints. Calling the low-level prefix builder without preparing its preconditions is not a complete motive-inference algorithm.

Implement a version-pinned adapter returning a structured result:

```
MotiveAttempt :=
  success(prefix, branches, localSubstitution, dependencies)
| unsupportedMajor(reason)
| indexPreparationNeeded(indices)
| typeMismatch(diagnostic)
| unresolvedConstraint(constraint)
| resourceExhausted
```

Run every attempt in a saved state with explicit heartbeat and recursion limits. Restore assignments, messages, local substitutions, and branch-local caches on failure. Do not parse fragile exception text as a proof of impossibility. Preserve the original diagnostic for developers while exposing a stable coarse category to the search.

This makes the initial indexed-induction improvement relatively contained: remove Leant2's blanket exclusion for a well-prepared subset of indexed families, reuse Lean's construction, and test the resulting branches. It does not solve general higher-order motive search, but it is a useful prerequisite before investing in a new motive language.

8 R5: General recursion from incomplete examples

8.1 R5.Q1. Does a capability interpreter agree with the realized recursive term definitionally?

Assessment: Not in general. Propositional simulation is the robust contract, especially for well-founded recursion.

Lean's documentation explicitly distinguishes structural recursion from well-founded recursion: well-founded definitions typically compute through propositional equation theorems rather than convenient definitional reduction, and kernel evaluation can be slow because it unfolds termination evidence [10]. Therefore the search interpreter should not promise that its execution trace is literally the kernel's conversion trace.

Define a typed body language with recursive calls admitted only through a capability

$$\text{rec} : (y : X) \rightarrow R(y, x) \rightarrow Y(y),$$

where a separate certificate establishes that R is well-founded. The body may call the capability at a provably smaller argument; the proof of decrease is part of the obligation, not an unchecked annotation. Realization constructs a structural or well-founded definition, and a theorem connects the body semantics to its equation lemmas.

A practical interpreter is fuelled and returns either a value or unknown:

$$\text{eval}_m(b, x) = \text{some}(y) \implies \text{realize}(b)(x) = y. \quad (7)$$

Prove this theorem by induction on the successful evaluation derivation, using the realization's equation theorem at each recursive step. Fuel exhaustion supplies no negative information. A natural-valued decrease measure may bound call depth, but not the entire number of reductions or the size of a branching recursion tree; define fuel precisely.

There are three evaluation profiles. In the narrowest profile, kernel reduction validates every observation directly. In a proof-producing profile, a fast interpreter supplies certificates used to rewrite the finite contract to decidable ground obligations. In a native profile, compiled evaluation may supply hints or rely on explicitly approved native-computation assumptions. Only the first two preserve a kernel-only computational trust boundary without additional assumptions. An axiom audit must reflect whichever profile is actually used; one should not describe native verification as if it were ordinary kernel reduction [11].

The proposal’s compilation defect reinforces this separation. A logical recursor term can pass the kernel yet fail the code generator’s expected representation. Keep the certified term, then construct a compilable equation form and prove its correspondence. A proof of logical correspondence and an explicit compiler trust policy are separate from successfully attaching an implementation annotation. Realization and certification must be charged to the reported end-to-end budget, not deferred beyond the advertised time to solution.

8.2 R5.Q2. What changes when angelic execution is combined with top-down search?

Assessment: The combination is coherent, but top-down search needs persistent relational assumptions and must revisit them when shared holes are filled.

Bottom-up synthesis has complete candidate fragments whose behavior can be summarized compositionally. A top-down skeleton can contain the same hole in several contexts, with type assignments still changing. Its angelic summary is therefore a relation on hole closures, recursive calls, and expected outputs, not just a set of values for one subtree.

Maintain an assumption store per branch. Give a recursive call a stable key containing the function being synthesized, typed argument values or symbolic arguments, and relevant parameters. Equal call keys share constraints. When unevaluation derives requirements for an unobserved call, record them in the store; when the call becomes executable, discharge them or refine the branch. When a shared hole changes, invalidate dependent assumptions as well as forward evaluations.

The safe initial version uses these assumptions to prioritize holes and candidate bodies but never prunes solely because one guessed execution fails. A stronger version may prune when a certified relational over-approximation has no satisfying assignment. This preserves the basic existential meaning of angelic execution and the conservative-refutation condition of Section 3. Burst supplies the main precedent, while Smyth shows the importance of interdependent goals and live evaluation [27, 15].

Recent work also broadens the comparison beyond examples. Cataclyst, described in a July 2026 preprint, combines sketches, enumerative search, mixed-quantifier first-order properties, and counterexample-derived syntactic constraints. Its setting and verification infrastructure differ from Lean’s, but it is a relevant source of refinements for recursive synthesis with relational contracts [31]. A counterexample-guided restriction imported from such a system still needs a Lean-side explanation of why it cannot eliminate a valid candidate in the represented grammar.

8.3 R5.Q3. Is there a small canonical basis of recursion schemes with measured coverage?

Assessment: There are effective published template bases, notably Para, but no unique small basis whose reported coverage transfers automatically to Leant2.

A paramorphism exposes both a child’s original input and its recursively computed result. For a polynomial functor F , its algebra has shape

$$\phi : F(\mu F \times K) \rightarrow K,$$

and the recursive computation replaces each child by the pair of that child and its recursive result before applying ϕ . An ordinary fold is the special case that ignores the original child. This gives one structured

interface for many apparently different recursive programs.

Para demonstrates synthesis using such templates and recursion-free hole fillers, including representations useful for multi-argument and higher-order computations [19]. Leant2 should adapt the template language rather than mechanically copy a surface syntax. The appropriate starting basis is structural recursion/paramorphism, bounded product and function-valued state, bounded nesting, and an explicit well-founded adapter. Tupled state handles Fibonacci; a function-valued state handles accumulator passing; one structural argument often suffices for zip; merge can decrease the sum of two list lengths; Euclidean gcd needs a modulus-decrease theorem, not merely an unconditional call to an arithmetic tactic.

A basis can be small because it hides large search problems in its algebras or termination proofs. Thus “one universal recursor” is not a useful answer to the coverage question. Measure both scheme coverage with ideal fillers and actual synthesis coverage under the same component inventory and resource budget. Missing comparisons, append, or arithmetic lemmas can otherwise be mistaken for missing recursion schemes.

Mutual and nested inductives should be handled through recursor metadata and Lean’s equation elaborator, not brittle rules based on generated declaration names. Initially admit a clearly documented subset and return unsupported for other forms. Helpers should be synthesized as parameterized subproblems with explicit call interfaces, avoiding hidden access to a parent’s changing local context. A bound of one helper is an experiment, not an expressiveness theorem. The roadmap should retain both carrier-given and carrier-invented benchmark tracks for every recursive family.

9 R6: Contracts beyond finite decidable observations

9.1 R6.Q1. How should value construction and proof obligations be interleaved?

Assessment: Use a dependency-aware obligation graph with cheap early propagation and delayed expensive proof search.

The dichotomy “prove immediately” versus “wait for a complete program” is too coarse. Some proof obligations can already determine data: solving $?n = 0$ by unification fixes a witness. Others are meaningful as symbolic local obligations under an induction hypothesis. Expensive general automation on a proposition whose relevant data holes are still unconstrained is usually wasteful, but postponing all reasoning loses valuable type information.

For each obligation, compute its support: the value holes, type assignments, local hypotheses, and instance choices on which it depends. Separate cheap propagation rules—reflexivity, constructor injectivity, contradiction, definitional simplification, and rigid index constraints—from open-ended proof attempts. Run the former when their premises are available. Schedule the latter after their support is stable, preferably at a branch boundary rather than only at the root.

Refinement-type synthesis supplies a useful conceptual precedent for using logical constraints while constructing terms [28]. Without an SMT solver, use a small finite vocabulary of predicates with Lean-checked implication certificates. This is not automatically a complete liquid-type inference procedure over arbitrary propositions; it is a restricted abstract domain whose transfer rules are justified in Lean.

For length-preserving map, synthesize a recursive value with target

$$\Pi xs : \text{List}A, \quad \{ys : \text{List}B \mid \text{length}(ys) = \text{length}(xs)\}.$$

The empty branch produces an empty list and a reflexive proof. The cons branch takes the recursive pair, constructs the new head and tail, and uses the recursive proof to discharge the successor-length equation. The proof is local and follows the program, rather than being reconstructed from an opaque completed recursor term.

This design needs a concrete implementation change. The inspected `proofPortfolio` currently rejects goals whose type contains free locals, not only unresolved metavariables. Local branch proofs therefore cannot simply be routed through that function unchanged. Introduce a contextual proof adapter using the goal’s local context, with support-aware caching and transactional rollback [4]. Completed proof terms may be cached only after abstracting the locals they use.

9.2 R6.Q2. Can one recognize properties provable by program-aligned induction and arithmetic?

Assessment: One can recognize a useful sufficient class by generating and classifying local verification conditions; a complete syntactic characterization is too strong.

Let a program be a fold $q(xs)$ with initializer z and step s . Supply an invariant relation $I(xs, k)$. Its proof obligations are

$$I([], z), \quad \forall a, xs, k, I(xs, k) \rightarrow I(a :: xs, s(a, k)). \quad (8)$$

An observation theorem then transfers $I(xs, q(xs))$ to the requested contract. Indexed constructors give the analogous family of branch obligations.

Theorem 9.1 (A certifiable program-aligned class). *Fix a recursion scheme with its induction principle, a finite checked rewrite system for constructor and program equations, and a sound decision procedure for a theory $\mathcal{D}$. If an invariant and its generated base and step obligations simplify to formulas accepted by $\mathcal{D}$, and the required observation theorem is proved, then the synthesized program satisfies the contract.*

Proof. The checked rewrites preserve each obligation. The decision procedure supplies proofs of all base and step obligations. Apply the recursion scheme’s induction principle to obtain the invariant at every input, then apply the observation theorem. Every composition is represented by an ordinary proof term. $\square$

This gives a positive recognizer: generate the obligations, perform the fixed rewrites, and check whether the residual syntax belongs to the supported theory. An initial theory can combine finite constructor case splits with linear arithmetic over naturals and integers. Length accounting and many simple measure bounds fit. Sorting additionally needs ordering and permutation invariants; an accumulator-based reverse proof often needs an associativity lemma and a generalized accumulator statement. Their failure to fall into the initial class does not establish that they need fundamentally different induction.

Membership in a narrowly defined residual theory can be decided syntactically. Whether *some* invariant, generalization, rewrite sequence, or library lemma would make an arbitrary Lean property fit is a new search problem. The engine should report “outside this certified induction tier,” not “not inductively provable.” Store successful invariants as reusable proof patterns, with their exact conditions, rather than expanding the default simplifier indiscriminately.

9.3 R6.Q3. What induction automation is usable in Lean now?

Assessment: Core induction and program-aligned proof construction are immediately usable; broader automated induction remains a portfolio component rather than a universal service.

For the pinned toolchain, `MVarId.induction` is an available meta-level library operation, while `simp`, `omega`, and `grind` can serve as branch dischargers. Lean’s library-search implementation itself exposes an example of invoking `Grind.main` on a goal with default parameters. This is a concrete integration path, not merely a suggestion to invoke an interactive tactic from a string [7, 8].

The current official manual, consulted as version 4.35.0-rc2, also documents `fun_induction`: it uses generated functional induction principles for non-mutual structurally or well-founded recursive definitions, supports generalizing, and can unfold the function call while setting up the proof. `fun_cases` provides the corresponding case analysis [12]. This is particularly relevant after publishing a candidate through the equation compiler. Check availability against Leant2’s pinned 4.34.0 build rather than silently treating the moving manual as that build.

Keep induction selection separate from branch automation. A program-guided adapter knows the recursive argument and the equations that generated the body; it can choose the corresponding induction principle and present the right local hypotheses. Calling `grind` on a closed theorem without that structure should not be assumed to discover the required induction and auxiliary invariant. Conversely, applying induction to every inductive local reintroduces the search explosion already observed in the proposal.

Canonical supplies an additional dependent inhabitation and recursion baseline, Aesop a rule-based proof-search framework, and LeanHammer a learned premise-selection plus proof-reconstruction approach [14, 13, 32]. These have different dependencies and trust boundaries. A core-only Leant2 default should use a small internal adapter and allow optional integrations through the same proof-service interface. No current benchmark for those tools establishes ten-second success on Leant2’s quantified synthesis contracts.

A useful service reply is a proof, a checked counterexample, an unsupported fragment, or resource exhaustion. Give each invocation a heartbeat quota and a cancellation boundary; an external process may additionally need an operating-system limit. A wall-clock deadline cannot necessarily interrupt all code at an arbitrary instruction. Record timeout separately from logical failure. Rippling-inspired generalization and lemma discovery should initially produce candidates for this service, not silently extend its trusted calculus.

9.4 R6.Q4. Is there a principled scheduler for proof debt?

Assessment: Yes as a constrained decision policy with explicit costs and fairness; no simple bandit index is universally optimal for these dependent obligations.

For an obligation o , estimate its remaining cost $\widehat{r}(o)$, chance of usefully resolving $\widehat{p}(o)$, and downstream benefit $\widehat{b}(o)$. A provisional priority is

$$\frac{\widehat{p}(o) \widehat{b}(o)}{\widehat{r}(o) + \widehat{s}(o) + \varepsilon}, \quad (9)$$

where $\widehat{s}$ models setup or switching cost. Benefit should include shared consequences: proving one lemma may discharge several candidates, while rejecting an early local invariant may eliminate a large subtree. Equation (9) is a heuristic whose assumptions are visible, not an optimality theorem.

These jobs are not stationary independent arms. Filling a data hole changes several proof problems; an arithmetic lemma can make all of them easy; a counterexample changes the synthesis frontier. An exact policy could be formulated over a finite state model with known transition probabilities and costs, but learning that model is itself expensive. Classic restart results likewise do not automatically cover this dependency structure [21].

Use a deterministic fair scheduler first: counterexamples before expensive proofs, local obligations before global reconstruction when they are ready, small proof quanta before larger ones, and a reserved share for synthesis. Attach each proof job to the support fingerprint on which it was created. Suspend rather than restart jobs when the solver permits it. Learn estimates only at logical epoch boundaries, and evaluate the learned policy against fixed round-robin and fixed-tier baselines. Measure total certified solutions and time to rejection, not the number of tactic calls that report success.

10 R7: Component retrieval at library scale

10.1 R7.Q1. What abstraction is useful for dependent, universe-polymorphic signatures?

Assessment: Use a refinable, constraint-carrying hypergraph; a graph of unrelated argument heads loses essential conjunctions and correlations.

Begin with a component descriptor containing its telescope, result head, rigid type-parameter sharing, selected index shapes, universe constraints, and class prerequisites. The first abstraction may erase most term indices while retaining facts such as a vector’s element type, whether an index is zero or a successor, and whether two occurrences share the same type parameter. A later abstraction can retain linear index expressions or a small predicate set. The purpose is to reject impossible compositions cheaply, not to solve full dependent unification outside Lean.

A component with two arguments is a hyperedge requiring both arguments, not two ordinary edges either of which enables its result. If values are reusable, model that reuse explicitly: a strict token-consuming Petri net without copying can exclude $x \mapsto (x, x)$. If the language permits unused inputs, do not silently impose the relevant-typing restriction that every input must be used. Retrieval and final ranking may prefer relevance without changing inhabitation.

Proposition 10.1 (The requirement on a retrieval abstraction). *If every well-typed composition in a candidate grammar maps to an abstract reachability derivation, then abstract unreachability excludes all such compositions. If a resource-limited abstract search merely fails to find a path, or if some constructors and recursion templates have no abstract counterpart, that conclusion does not follow.*

Proof. A concrete composition would supply an abstract derivation by hypothesis, contradicting abstract unreachability. A timeout is not unreachability; a missing abstract counterpart invalidates the hypothesis. $\square$

TYGAR’s abstract typing and refinement framework is the relevant precedent [16]. In Lean, a failed concrete elaboration is not automatically a proof of untypeability: higher-order unification can be incomplete, and instance resolution or normalization can time out. Refine from a definite rigid constructor clash, a checked incompatible index constraint, or another justified obstruction. Otherwise deprioritize the candidate and retain the possibility of a different instantiation.

Use universe constraints explicitly or conservatively forget them. Rejecting an abstraction because a guessed universe assignment failed is unsound when another assignment could work. Likewise, a path-length cutoff of three is a bounded retrieval heuristic, not evidence of general uninhabitability. Keep a fair fallback provider stream if completeness relative to a larger grammar is claimed. Store indexes by environment fingerprint and declaration identity, not only a namespace or package version string.

10.2 R7.Q2. Has type-guided abstraction refinement been combined with type classes?

Assessment: Yes. Hoogle+ uses dictionary passing directly; Lean adds contextual and dependent-instance complications.

The Hoogle+ paper’s Section 2.5 translates a class requirement such as `Eq a` into an ordinary dictionary argument. A transition requiring comparison is enabled only if the dictionary type is inhabited. Instance constructors can themselves provide dictionaries. The answer to the literal question is therefore already positive, not an unexplored extension of TYGAR [16].

For Lean, retain a class requirement as an explicit prerequisite in the component descriptor, and resolve it when enough type information is known. A dictionary supplied as a local argument is an ordinary available

value. An inferred dictionary must be justified by Lean’s instance mechanism or an explicit construction admitted by the query’s grammar.

Memoizing only by “class and type” is insufficient. The full instance goal can contain multiple parameters, indices, output parameters, and universes. Local instances and their priorities can change the answer. An unresolved metavariable may later be instantiated in a way that enables a previously failing search. Key a resolved result by the normalized complete goal, relevant local-instance context, environment, and assignment fingerprint; revalidate it on replay. Cache timeout as a resource event, not nonexistence.

Most importantly, computational dictionaries are not proof-irrelevant. Two Add A instances can produce different programs; two equality implementations can yield different observations. Their actual identity and behavior belong in the candidate and its observational cache. Only a proposition-valued field enjoys the corresponding proof-irrelevance guarantee. This is why type-class caching and candidate equivalence must share a semantic identity policy instead of independently erasing all instance arguments.

10.3 R7.Q3. Can existing premise retrieval support data goals and examples, and where should training data come from?

Assessment: Data-goal lookup is already possible; example-conditioned program relevance needs a separate adapter and an appropriate benchmark.

The pinned Lean library-search source includes a data-goal example, `example : Nat := by exact?`. Thus basic retrieval is not intrinsically restricted to propositions [8]. However, accepting a goal metavariable is not the same as learning how candidate functions transform example inputs. A premise selector trained on theorem proofs may offer useful symbol overlap while missing the relevant behavior of a polymorphic function.

A Leant2 retrieval adapter should combine a type-shape query with a separate behavioral sketch: available input types, observed calls, output constructors, size changes, and simple relations such as preserving length. The existing selector can supply one ranked source. A deterministic symbolic index supplies another. A behavior-aware model, when available, supplies a third. Do not claim that passing extra prose to a theorem retriever gives a validated example-conditioned ranking interface.

Recent retrieval work, including LeanHammer and LeanSearch v2, improves premise retrieval for theorem proving, but its reported evaluations are not measurements of carrier- or example-conditioned program retrieval [32, 33]. Hoogle+’s published evaluation uses hundreds of components, so it should not be cited as direct experimental evidence for Leant2-scale retrieval across every imported declaration [16].

A useful program corpus can be constructed from small safe definitions in Init, Std, and selected Mathlib modules. Extract types, explicit component dependencies from elaborated bodies, and generated observations where the domains are executable. Train on component use, not merely names occurring in pretty-printed text. Remove the target definition from the provider set and remove target-specific equation lemmas or wrappers that trivially reveal its body. Group near-duplicate algorithms and their lemmas into the same split; respect import and source-order availability.

Maintain two evaluation tracks: API composition, where existing implementations may legitimately be used, and body synthesis, where the target and equivalent wrappers are forbidden. Report retrieval recall of a usable component set and end-to-end certified synthesis separately. Compiled behavioral probes can improve ranking, but failure of a standalone component on a whole-function example cannot refute its usefulness inside a composition.

11 R8: Ranking and candidate equivalence

11.1 R8.Q1. What formal objective is defensible for ranking inhabitants?

Assessment: A conditional description-length or Bayesian objective is defensible; agreement with human intent remains empirical and task-dependent.

Choose a proper grammar-based prior $P_\theta(t \mid T, \Gamma, E)$ and rank contract-satisfying terms by

$$J(t) = -\log P_\theta(t \mid T, \Gamma, E) + \lambda_r \widehat{\text{runtime}}(t) + \lambda_p \widehat{\text{proofCost}}(t). \quad (10)$$

The first term is a description length under a declared coding model. The other terms express engineering preferences, not logical correctness. A frozen prior makes the objective reproducible. A changing exploration model need not change J .

This objective clarifies why a reusable library call can be better than a large reconstructed implementation, and why changing the library changes the ranking. It also exposes the limitations: a contract with two examples may admit several sensible programs, and no distribution-free rule identifies the user’s intended one. A studied synthesis system’s “interesting results” or reference-solution ranks are not equivalent to a controlled user-preference study. Hoogle+, for instance, explicitly uses relevance filtering and manual judgment in its evaluation [16].

Treat unused inputs as a soft prior, not a universal defect. A constant function can be exactly right. A proof argument may be computationally irrelevant. Syntactic occurrence is also not semantic influence: an input can appear in $x - x$ while not affecting the output. Exact semantic influence is not generally cheap to decide. Approximate it using normalized syntax and probe perturbations, labeling it as a heuristic.

Prefer one readable representative of each observed behavior, show a separating input when candidates differ, and preserve an expandable list of alternatives. A reference solution’s rank is useful for regression testing, but should be measured both by exact syntax and by an appropriate behavioral or proved-equivalence class.

When a frontier justifies stopping. Let $L(s)$ be a lower bound on the final *same* objective J for every completion of frontier state s . If every unexplored alternative, pending proof job, and delayed retrieval choice is represented, and

$$\min_{s \in \mathcal{F}} L(s) \geq J(t_*),$$

then no strictly better candidate remains. This follows immediately by applying the bound to the frontier ancestor of any hypothetical better completion. A strict inequality also excludes an unseen equal-cost tie. Actual CPU work already spent is not a lower bound on description length; nor are stale learned queue scores. The proposal’s stopping argument is valid only after those distinctions are enforced.

11.2 R8.Q2. Which candidate equivalence is cheap and meaningful for dependent types?

Assessment: Use several explicitly named equivalence levels, not one catch-all normalization or observational hash.

Start with alpha-aware structural identity on elaborated expressions; pretty printing is not an adequate key. Next use checked definitional equality in a transactional context. A stronger merge can use a checked propositional function equality, or a typed heterogeneous relation where required. The proof and any necessary transports must remain available to reconstruct the claimed substitution of one candidate for another.

For presentation, define the finite observational signature

$$\text{sig}_{\mathcal{P}}(t) = (\text{observe}_p(t))_{p \in \mathcal{P}},$$

where each probe is typed and may return unknown. Equal complete signatures mean “agrees on these probes,” not global equivalence. For dependent results, the probe must specify how to compare values in the relevant fibers; raw printed-value equality is insufficient. Unknown observations should not be treated as evidence of equal values.

Observational buckets are safe as a display device. They are not generally safe as permanent search pruning. Two functions may agree on the current input 0 and differ when a later context applies them at 1. If new tests arrive, retain enough alternatives to refine the buckets. In bottom-up synthesis, substitution by a representative requires congruence for every future context admitted by the observation semantics; it is not guaranteed merely because the subterms are closed.

Within an audited parametric fragment, Section 5’s canonical tests give stronger length-bounded guarantees. Outside it, distinguish evidence levels in the interface. Proof irrelevance may identify proof fields, but computational dictionaries and nontrivial dependent casts must not be erased from the semantic key. This policy avoids both duplicate clutter and false claims that finitely observed candidates are the same program.

12 R9: Learned guidance under a kernel gate

12.1 R9.Q1. What accuracy do current models achieve on accumulator or helper proposals?

Assessment: No numerical estimate is justified for this exact Leant2 task by the evidence examined. A targeted study is necessary.

Whole-proof success, general code generation, and component prediction are not measurements of producing the right carrier or helper from only a Lean type and examples. A March 2026 study compares language models with symbolic tools across several synthesis domains, including recursive functions, and reports important accuracy and runtime limitations. Its prompts, models, specifications, and verifiers differ from the proposed carrier-proposal task [34]. It cannot supply a percentage for Leant2 by extrapolation.

A focused experiment should separate four questions. Does the proposal parse and type check? Does it contain a carrier or helper with a solution under an ideal or supplied filler? Can the actual bounded synthesizer find that filler? Does the resulting complete program satisfy the contract with a checked proof? These are nested but different success rates. A good carrier can appear unsuccessful because the step search is weak; an apparently successful helper can overfit sparse examples.

Use held-out algorithm families rather than randomly split variants of the same textbook function. Vary example completeness, noise-free ambiguity, and whether a sketch is supplied. Compare the same frozen enumerator with no model, model-based reordering, and model-supplied additional templates. Record top-one and top- k useful-template recall, invalid proposals, time to certified answer, model startup and inference latency, and replay success. Charge all model work to the same end-to-end budget, including a cold-start track.

One proposal batch per query is a better initial experiment than a model call at every node. Carrier, generalization, and helper-shape decisions have large possible downstream effects, whereas a call for every routine constructor would pay latency repeatedly. This is a hypothesis about where guidance may be economical; the proposed study should be allowed to reject it.

12.2 R9.Q2. Can learned contributions be deterministic and auditable beyond recording proposals?

Assessment: Yes through frozen artifacts and a restricted interface; a seed and temperature zero are not a general reproducibility guarantee.

Separate three notions. *Proof replay* rechecks a final term against the original type and contract. *Search replay* repeats an exact sequence of proposals and decisions. *Model replay* regenerates identical proposals from the model. The first two can be achieved without the third by recording proposal bytes and a deterministic logical trace. Floating-point execution, parallel inference, and changes in libraries or hardware can affect the third even when a seed is fixed.

A receipt should identify the query and original contract, environment and provider inventory, Lean toolchain, trust profile, search grammar, cost model, proposal bundle, logical work counters, and accepted program and proof. For a learned source also record model and tokenizer hashes, prompt/template version, inference implementation, and any normalization of its output. An offline content-addressed proposal bundle or a frozen integer-valued ranking table is particularly easy to audit.

Restrict proposals to a small typed syntax or a list of named components and binder selections. Arbitrary Lean commands, tactic blocks, and imports can execute metaprograms; the phrase “the model only proposes Lean” is not a sandbox. Elaborate untrusted syntax in an isolated environment, then validate the resulting term against the independently retained query. Lean’s current validation documentation explicitly distinguishes ordinary kernel checking from protection against malicious code and describes statement comparison and external checking in a sandboxed workflow [11].

Reordering preserves completeness only with the right quantifiers. Exhaustive search over a fixed finite structural bound still sees the same candidate set if every alternative is eventually processed and pruning remains conservative. A fixed time or node budget need not see the same set after reordering. An infinite search needs fairness. If the model adds a new helper or component, it has changed the grammar, not merely reordered it, and the receipt should say so. Reserve a deterministic fallback share and never turn a zero model probability into a semantic impossibility claim.

For correctness citations, the authoritative artifact is the checked program, its exact contract proof, and its allowed assumptions. The model’s ranking score is not evidence of correctness. Recording why the search tried a candidate is useful for engineering audit, but cannot replace replay of the proof or a certificate for a claimed negative result.

13 An integrated implementation plan

The answers above point to a staged architecture, not a replacement of every working subsystem at once. The most valuable immediate work makes existing capabilities composable: local proof goals, indexed induction, callable library retrieval, explicit observations, and correct persistence of branch state. The more speculative parts—general lifting, symbolic higher-order unevaluation, and learned proposals—then become extensions of a stable interface rather than competing search engines.

13.1 Four interfaces with different authority

The following is interface-level pseudocode, not a compiled Lean implementation. A `QueryId` identifies the original type, contract, imported environment, local telescope, and trust policy. It must not be reconstructed from a model’s output or from a pretty-printed candidate.

```

ProposalService.propose : DecisionPoint -> SearchView -> Proposals
ObservationService.check : PartialProgram -> ObservationReport
ProofService.tryProve : TypedObligation -> Budget -> ProofReply
AcceptanceGate.check : OriginalQuery -> Candidate -> Proof -> GateReply

```

The proposal service can suggest terms, carriers, generalizations, components, or guards. Its outputs carry no authority to remove alternatives. The observation service can return values and blockers without claiming a proof; a pruning result additionally carries a certificate or invokes a documented, conservative evaluator. The proof service distinguishes a checked proof from an inconclusive attempt. The gate reconstructs the exact requested statement, checks its proof, and audits the resulting assumptions.

Every heuristic service should have a bounded deterministic fallback. This preserves the project’s no-settings, core-only default: internal versioned policies need not become a collection of user-facing switches. A sketch is still a valuable way to express knowledge about a carrier or helper; it changes the mathematical problem rather than tuning an opaque engine.

13.2 State ownership and a resumable transition system

A frontier node should contain the saved meta state, typed root expression, open obligations, dependency graph, residual-cache references, recursion capabilities, accumulated immutable ranking information, and an expansion cursor. The cursor is essential: a rule producing many alternatives must be suspendable without repeating all its earlier work. A node also records its logical predecessor and the rule instance that produced it.

Global monotone data include work accounting, checked closed counterexamples, and successful reusable lemmas whose dependencies are explicit. A cached residual fact concerning an open term is not global merely because it lives in an `IO.Ref`. Separate reusable content-addressed entries from branch-local views of those entries. The existing snapshot machinery is a useful base, but its external references require this ownership audit [5, 4].

The initial implementation can preserve the current depth-first order while exposing one transition at a time. Verify that this refactoring reproduces the old logical trace on deterministic-work fixtures before changing scheduling. Then add a persistent priority queue, with memory caps and an explicit spill, eviction, or resumable-depth-first policy. Eviction must not masquerade as exhaustion. Successful snapshots being cheap on one profile does not imply that retaining millions of them is cheap.

```

while resources remain:
  job := deterministicScheduler.next()
  restore(job.branchState)
  validateDependentCaches(job)
  result := advanceOneLogicalQuantum(job)
  match result:
    proposal(children): enqueueWithStableIds(children)
    blocked(dependencies): suspendUntilRelevantChange(job)
    refuted(certificate): checkAndRecord(certificate)
    completed(term): scheduleContractAndPublicationChecks(term)
    proofReady(term, proof): gate(originalQuery, term, proof)
    unknown(reason): retainOrSuspend(job, reason)
  commitTraceEvent()
return certifiedAnswers, frontierReceipt, incompleteJobs

```

A logical quantum is a bounded unit of resumable work, not necessarily one Lean primitive. A primitive that

can perform substantial reduction needs its own resource boundary. Wall-clock control remains a safety limit around the logical schedule, not its definition.

13.3 Recommended order of implementation

Stage A: preserve the boundary and remove integration bottlenecks. Harden the public gate against mismatched supplied proof statements. Add local context support to the proof-service adapter; the current closed-proposition portfolio explicitly excludes free variables, which would block the branch proofs needed by program-and-proof synthesis. Reuse Lean’s indexed-induction operations on their supported inputs, add elementary natural-number recursion under explicit recursion budgets, and make single-step library lookup available to typed holes. Separate a certified kernel term from a compilable, re-elaborated presentation [6, 4, 7, 8].

Acceptance gates for this stage should be concrete: vector map with its given motive, a proof involving a local induction hypothesis, the missing elementary Nat lemmas, and a published recursive candidate that both rechecks and runs. Failure of publication must preserve the certified result while clearly reporting that executable publication failed. Do not use an unchecked compiled replacement to conceal the discrepancy.

Stage B: observations and branch-safe reuse. Split supported Boolean and propositional contracts into typed observations, retaining the reconstruction theorem. Store per-observation blockers and assignment fingerprints. Add checked counterexamples and test them early on later candidates. Introduce a simple observation record for closed first-order calls before general hole closures. Differential tests against the current kernel path are useful engineering evidence, but any stronger claim of conservative pruning must name its proof or trusted implementation.

A useful milestone is fewer repeated evaluations of unchanged observations without changing the bounded candidate set. Measure invalidations and cache hits as well as elapsed time. A cache that seems fast only because it misses necessary invalidations is a regression, not an optimization.

Stage C: bounded lifting with a carrier-given control. Implement one explicit grammar containing result state, one extra parameter, a product with one auxiliary quantity, and optional state. Include observer and step obligations from the start. Run every benchmark twice: with the reference carrier supplied, and with that carrier hidden. When a problem fails even with the carrier supplied, improve body construction or proof support before blaming carrier invention.

Add a finite-state conflict lower bound only where carrier cardinality is certified. For symbolic or polymorphic state, use conflicts to propose useful features, not to count values already enumerated. Connect lifting templates to the induction invariant interface, so a synthesized auxiliary quantity comes with a path to certification. This stage can benefit from the ideas in AutoLifter and Para without implementing their full search machinery [17, 19].

Stage D: change search order after exposing resumable transitions. Introduce the immutable cost model, deterministic tie-breaking, and a fair baseline frontier. Start with zero or a proved relaxed lower bound. Add goal ordering based on estimated domain size and dependency impact. Next introduce independent-component solution caches; delay general interface-relation tabling until simpler caches show a benefit. A successful best-first rewrite must justify its memory use and candidate improvements, not merely eliminate iterative deepening from the source code.

Stage E: stronger recursion and proof-producing evaluation. Add a typed recursive body representation and explicit realization obligations. A structural or well-founded call capability must have a checked decrease proof. Implement a small supported evaluator with certified output equality, then extend it with typed hole closures. Unresolved recursive values remain symbolic and shared across equal calls. Add angelic constraint refinement only when the store can distinguish an impossible relation from a failed guess.

For universal contracts, first implement the induction-aligned verification conditions described in R6.Q2. A generated invariant template that fails is an invitation to search for a stronger invariant, not a reason to reject the program globally. Keep unsupported proof obligations visible in the outcome.

Stage F: learned proposals only after the controls exist. Evaluate fixed, recorded carrier and helper proposals against enumeration with the same grammar, proof budget, and full latency accounting. Add live inference only after this offline comparison shows a useful increase in certified solutions. A local model is optional; it must not become a hidden dependency of the core-only path. External proof tools likewise enter through explicit adapters with recorded assumptions and resource limits.

These stages express dependencies, not calendar estimates. Stages A and B should proceed without waiting for a universal carrier-minimization algorithm. The decision to continue each later stage should be based on the ablations below, rather than on novelty alone.

14 Evaluation, falsification, and reproducibility

14.1 A benchmark design that diagnoses causes

Keep the reported scored suites as regression fixtures, preserving their outcome-category definitions. Report the thirteen unscored stretch cases as a separate track rather than quietly adding them to a previously saturated score. The edge probes should also remain visible by name. Imported literature corpora need deduplication by semantic task family: reverse with three slightly different examples is not three independent demonstrations of generalization. The repository’s current reports are historical comparison points, not data reproduced in this study [2, 3].

For each task, record the target, contract, providers, allowed recursion schemes, reference carrier or motive, proof strategy, and trust profile. Create paired conditions that isolate the hard part: type only versus finite examples versus a universal property; carrier supplied versus invented; helpers supplied versus retrieved versus invented; trace-complete versus incomplete examples; and raw kernel term versus executable published result. This prevents an easy provider lookup from being counted as evidence of recursive body synthesis.

Measure time to first certified answer, time to executable publication, rank of a reference-equivalent answer, number of distinct observed behaviors, number of checked refutations, peak memory, and total logical work. Report kernel checking and proof reconstruction separately but also include them in end-to-end latency. Report both a cold-start path and an already-loaded interactive path. Model calls, index loading, and compilation must not vanish from a ten-second claim merely because they run in a different process.

Timed measurements should fix hardware, imported environment, toolchain, thread count, and cache policy. Repeat runs to show variation. Deterministic logical-work measurements should compare exact trace prefixes and candidate sets at a fixed completed prefix. These are complementary experiments; neither should be substituted for the other.

Table 1: Ablations and the hypotheses they can actually test.

Change	Compare against	Primary question
Observation cache	Cache disabled, identical expansion order	Are repeated reductions avoided without changing decided outcomes?
Queue scheduler	Resumable depth-first baseline	Does better order outweigh queue and snapshot memory?
Goal ordering	Fixed order; same rule grammar	Does dependency-aware ordering reduce branching rather than merely change answers?
Carrier inference	Carrier supplied; fixed grammar enumeration	Is carrier discovery, body filling, or proof construction the actual bottleneck?
Angelic store	Unknown calls remain stuck	Does refinement help incomplete examples while preserving bounded alternatives?
Inductive proof tier	Closed-candidate proof portfolio	Do local obligations increase certified, not just tested, successes?
Library retrieval	Curated providers; full bounded fallback	How much branching is removed, and which valid components are missed?
Learned proposals	Recorded proposals shuffled or omitted	Does guidance improve certification after its complete cost is charged?

14.2 Adversarial tests for the claimed guarantees

A correctness suite should deliberately attack the interfaces. Use two metavariable states with identical printed targets but different universes or instance dictionaries. Assign a hole, evaluate, undo, and assign a different value under the same identifier. Exercise delayed assignments whose local contexts differ. Introduce an observation joining previously independent components. Request a proof of a contract different from the supplied proof type. Give a custom BEq that is not lawful equality. Offer two programs that agree on the current probe set and disagree on one new input.

For recursion, use repeated equal calls in a constraint that demands inconsistent outputs, and a case where one angelic assignment fails while another works. Set evaluator fuel just below the amount required by a correct candidate and check that the result is unknown rather than false. Distinguish conversion failure, proof-search failure, and unsupported realization. For ranking, keep an unexplored state whose correct lower bound permits an answer better than the incumbent; verify that no optimality claim is issued.

Every such test needs two checks: positive answers still pass the exact gate, and pruning does not remove a known valid bounded completion. The second check is essential because a perfectly sound acceptance gate cannot detect a solution that the search discarded prematurely.

14.3 Checks actually executed for this article

The package includes `checks/check_models.py`, a standard-library Python program, and its recorded `results.json`. All eleven checks passed. They are finite mathematical sanity checks and small exhaustive searches, not tests of Lean, Leant2, the proposed APIs, or runtime performance.

The largest exhaustive negative check enumerates every one-state and two-state deterministic right-fold machine over a binary alphabet with a Boolean observer, fixing the initial state to zero without loss under

state renaming. On all 31 words of length at most four, the requested output is true exactly when length is divisible by three. All 2 one-state tables and all 64 two-state tables fail. The three-state search finds a witness after 3,173 tables; it does *not* exhaust the three-state space. The witness cycles through residues 0, 1, 2 and observes whether the state is zero. The three-state lower bound for arbitrary lengths is the context-distinction argument in R2.Q1, not an inference from testing alone.

Other checks evaluate the ordinary reversal fold on 1,093 words, check the Fibonacci pair invariant for 31 inputs, and exhibit counterexamples to summed coupled-goal bounds, incomplete-row state counting, assignment-blind rollback caching, finite-probe equivalence, independent choices for repeated angelic calls, and machine-independent wall-clock answers. Their purpose is to make the article’s warnings reproducible. They do not replace the written proofs or the implementation tests specified above.

15 Conclusions

The expert questions are productive because they identify interfaces at which otherwise successful methods fail to compose. The strongest answers are not all new algorithms. Some are existing capabilities that should be reused: higher-order live examples, dictionary-passing retrieval, auxiliary-function synthesis, and Lean’s motive-construction and induction operations. Others are corrected specifications: finite-state minimality is not minimality of a Lean type; a checked positive answer is not an exhaustive search; and reproducible work is not identical work completed before every deadline.

The proposed path is therefore to build a bounded, explicit, evidence-carrying system first. Keep the kernel gate, strengthen its connection to the original query, expose resumable transitions, make dependencies and unknown outcomes visible, and synthesize lifting and recursion templates with local proof obligations. Only then let more ambitious evaluators, tabling schemes, and models accelerate the same well-defined problem. The main unresolved empirical question is how much each addition increases useful certified coverage per unit of end-to-end cost. The benchmark design above makes that question answerable without conflating promising examples with established performance.

A Question-by-question coverage

Question identifiers preserve the proposal’s research-area numbering and the order of questions within each area. R8 and R9 are both in its sections/12-ranking.tex. The consolidated expert-contact section reorders these themes; it does not add a separate set of research questions. The summaries below paraphrase the questions rather than reproduce them.

Table 2: All 31 expert questions and the answer supplied.

ID	Question theme	Main answer	Page
R1.Q1	Shared metavariables	Conditional factorization, persistent branches, interface-aware tabling.	6
R1.Q2	Admissible remaining cost	Maximum of valid bounds; sums only with cost separability.	7
R1.Q3	Online learning and ranking	Freeze the objective; adapt priorities with recorded logical epochs.	7
R1.Q4	Deadline determinism	Identical timed answer sets are generally impossible; replay prefixes.	8

ID	Question theme	Main answer	Page
R2.Q1	Minimal carrier type	Semantic quotient and finite-state lower bounds; bounded syntax search.	9
R2.Q2	Polymorphic alphabet	Naturality yields position tests under explicit assumptions.	10
R2.Q3	Backward fusion	Use lifting and auxiliary-function precedents with invariant obligations.	10
R2.Q4	Recursor motive basis	Bounded typed carrier/motive grammar, including parameter binders.	11
R2.Q5	Generalization heuristics	Generalize changing state and synthesize strengthening candidates.	12
R3.Q1	Higher-order/live evaluation	Smyth already supports higher-order examples; dependent fibers add work.	12
R3.Q2	Incremental normalization	Dependency read sets with rollback-safe assignment identities.	13
R3.Q3	Conservative fast evaluator	A typed fragment with checked evaluation/refutation evidence.	14
R3.Q4	Top-down angelic guarantees	Finite bounded completeness under decisive constraints and fairness.	14
R4.Q1	Motive enumeration	Finite abstraction templates plus bounded accumulator closure.	15
R4.Q2	Canonical transports	Normalize indices with evidence; do not erase required casts.	15
R4.Q3	Unfolding policy	Staged head exposure informed by Canonical and documented sauto options.	16
R4.Q4	Lean motive API	Reuse prefix construction and induction with checked preconditions.	16
R5.Q1	Interpreter versus realization	Propositional simulation and equation proofs; fuel exhaustion is unknown.	17
R5.Q2	Top-down angelic composition	Shared typed call constraints and explicit assumption discharge.	18
R5.Q3	Small recursion basis	Paramorphism-oriented templates; measure coverage by capability.	18
R6.Q1	Value/proof interleaving	Dependency-aware readiness and local invariant verification conditions.	19
R6.Q2	Recognizable induction class	Sufficient syntactic residual-theory tests, not universal recognition.	20
R6.Q3	Usable induction automation	Core meta APIs and functional induction, with bounded branch solvers.	20
R6.Q4	Proof-debt scheduling	Deterministic fair quanta plus cost/benefit estimates.	21
R7.Q1	Dependent type abstraction	Constraint-carrying hyperedges with a proved over-approximation.	22
R7.Q2	Classes in refinement nets	Hoogse+ dictionary passing; preserve Lean's contextual dictionaries.	22
R7.Q3	Program relevance learning	Data-goal lookup exists; build example-aware retrieval with clean splits.	23

ID	Question theme	Main answer	Page
R8.Q1	Ranking objective	Explicit description-length prior and evaluated user-facing preferences.	24
R8.Q2	Candidate equivalence	Separate definitional/proved equality from provisional probe buckets.	24
R9.Q1	Model carrier accuracy	No directly transferable number; measure four separate success rates.	25
R9.Q2	Deterministic auditability	Separate proof, search, and model replay; record restricted proposals.	26

B Source audit and scope of verification

The repository reading was pinned to commit 784664ff34e819422510a4d470ab07fe48bb736b, dated September 18, 2026. All nine research areas and their questions were read in their source TeX, together with the evidence, engineering, adaptation, and consolidated expert sections, the bibliography, the root README, and the implementation notes. The analysis of current code uses selected modules rather than a claim to have audited the entire repository. Stable source URLs are in the bibliography.

Source	What was checked
Leant2/Search/Core.lean, lines 1–440	Provider indexing, local frontier, snapshots, delayed substitution, residual decisions and blockers, and the closed-goal proof portfolio.
Leant2/Native/Transaction.lean	Search-context references, work accounting, deadlines, and rollback behavior.
Leant2/Accept/Gate.lean	Joint universe generalization, synchronous checking, supplied proof type, and axiom audit.
Lean 4.34.0: src/Lean/Meta/Tactic/Induction.lean	Index preconditions, motive prefix, dependency reversion, and returned branch substitutions.
Lean 4.34.0: src/Lean/Meta/Tactic/LibrarySearch.lean, lines 1–80	Data-goal example and direct Grind.main adapter.
Official Lean manual, current version shown as 4.35.0-rc2	Functional induction, recursive definitions, and proof-validation guidance; not treated as pinned 4.34.0 source.

Primary papers were used for the claims about existing synthesis methods. Where a current work is a preprint, it is identified as such; its results are not promoted to a theorem about Leant2. Bibliographic details in the original proposal were not simply copied: the investigation specifically checked the features relevant to each question. Absence of a directly matching benchmark in the sources examined does not prove that no such experiment exists.

No Lean executable was available in the working environment, and the Leant2 baseline and proposed adapters were not run. The only executable evidence newly produced here is the finite Python suite described in Section 14. Ordinary mathematical proofs and implementation recommendations must not be represented as a completed Lean formalization.

C Using the accompanying package

Build the article from the package root with

```
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

A standard TeX distribution with the listed packages is sufficient; no bibliography service, external figure download, or font file is required. The bibliography is included as `references.tex`. Without `latexmk`, run `pdflatex` three times so the contents and references settle.

Run the finite-model checks with Python 3.9 or newer:

```
python checks/check_models.py --output checks/results.json
```

Use a normal Python invocation without `-O`, because the checks use assertions. The script needs no third-party package. Its JSON output records which cases were examined and the discovered finite-state witness.

The package also includes a README and a provenance manifest. The manifest records the repository commit and the distinction between inspected code, published results, written mathematical arguments, and executed finite checks. It is a provenance aid, not a cryptographic certificate of the article's mathematical conclusions.

References

- [1] Vladimir Reshetnikov. *Leant2 repository and README*. Commit 784664ff34e819422510a4d470ab07fe48bb736b, September 18, 2026. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b>.
- [2] Leant2 project. *Leant2: Design for Further Improvements*. September 18, 2026. Source at the pinned repository commit, including sections/05-search.tex through sections/12-ranking.tex. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements>.
- [3] Leant2 project. *Implementation notes*. September 18, 2026. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/docs/implementation-notes.md>.
- [4] Leant2 project. *Construction search*, Leant2/Search/Core.lean. Inspected lines 1–440, pinned commit. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Search/Core.lean>.
- [5] Leant2 project. *Transactional search state*, Leant2/Native/Transaction.lean, pinned commit. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Native/Transaction.lean>.
- [6] Leant2 project. *The acceptance gate*, Leant2/Accept/Gate.lean, pinned commit. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Accept/Gate.lean>.
- [7] Lean contributors. *Meta-level induction implementation*. Lean 4.34.0, src/Lean/Meta/Tactic/Induction.lean. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Induction.lean>.
- [8] Lean contributors. *Library search implementation*. Lean 4.34.0, src/Lean/Meta/Tactic/LibrarySearch.lean, inspected lines 1–80. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/LibrarySearch.lean>.
- [9] Lean contributors. *Basic natural-number theorems*. Lean 4.34.0, src/Init/Data/Nat/Basic.lean, especially the addition theorems. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Init/Data/Nat/Basic.lean>.
- [10] Lean contributors. *The Lean Language Reference: Recursive Definitions*. Current manual consulted September 21, 2026 (version shown: 4.35.0-rc2), especially well-founded recursion. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.
- [11] Lean contributors. *The Lean Language Reference: Validating Proofs*. Current manual consulted September 21, 2026. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [12] Lean contributors. *The Lean Language Reference: Tactic Reference*. Current manual consulted September 21, 2026 (version shown: 4.35.0-rc2), entries fun_induction and fun_cases. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [13] Jannis Limperg and Asta Halkjær From. Aesop: White-Box Best-First Proof Search for Lean. *CPP*, 2023. <https://doi.org/10.1145/3573105.3575671>. Author version: <https://people.compute.dtu.dk/ahfrom/aesop-camera-ready.pdf>.
- [14] Chase Norman and Jeremy Avigad. Canonical for Automated Theorem Proving in Lean. *ITP*, LIPIcs 352, article 14, 2025. <https://doi.org/10.4230/LIPIcs.ITP.2025.14>. <https://arxiv.org/abs/2504.06239>.

- [15] Justin Lubin, Nick Collins, Cyrus Omar, and Ravi Chugh. Program Sketching with Live Bidirectional Evaluation. *Proceedings of the ACM on Programming Languages* 4 (ICFP), article 109, 2020. <https://doi.org/10.1145/3408991>. Author version: <https://jlubin.net/assets/icfp20.pdf>.
- [16] Zheng Guo, Michael James, David Justo, Jiaxiao Zhou, Ziteng Wang, Ranjit Jhala, and Nadia Polikarpova. Program Synthesis by Type-Guided Abstraction Refinement. *Proceedings of the ACM on Programming Languages* 4 (POPL), article 12, 2020. <https://doi.org/10.1145/3371080>. Author version: https://ranjitjhala.github.io/static/google_plus.pdf.
- [17] Ruyi Ji, Yuwei Zhao, Yingfei Xiong, Di Wang, Lu Zhang, and Zhenjiang Hu. Decomposition-Based Synthesis for Applying Divide-and-Conquer-Like Algorithmic Paradigms. *ACM Transactions on Programming Languages and Systems*, 2024. <https://doi.org/10.1145/3648440>. Version consulted: <https://arxiv.org/html/2202.12193v4>.
- [18] Shingo Eguchi, Naoki Kobayashi, and Takeshi Tsukada. Automated Synthesis of Functional Programs with Auxiliary Functions. *APLAS*, pp. 223–241, 2018. https://doi.org/10.1007/978-3-030-02768-1_13.
- [19] Qiantan Hong and Alex Aiken. Recursive Program Synthesis using Paramorphisms. *Proceedings of the ACM on Programming Languages* 8 (PLDI), article 151, 2024. <https://doi.org/10.1145/3656381>. Author version: <https://theory.stanford.edu/~aiken/publications/papers/pldi24.pdf>.
- [20] Shraddha Barke, Hila Peleg, and Nadia Polikarpova. Just-in-Time Learning for Bottom-Up Enumerative Synthesis. *Proceedings of the ACM on Programming Languages* 4 (OOPSLA), 2020. <https://doi.org/10.1145/3428295>. <https://arxiv.org/abs/2010.08663>.
- [21] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters* 47(4):173–180, 1993. [https://doi.org/10.1016/0020-0190\(93\)90029-9](https://doi.org/10.1016/0020-0190(93)90029-9).
- [22] Jeremy Gibbons, Graham Hutton, and Thorsten Altenkirch. When Is a Function a Fold or an Unfold? *Electronic Notes in Theoretical Computer Science* 44(1), 2001. <https://www.cs.ox.ac.uk/publications/publication2368-abstract.html>.
- [23] Jean-Philippe Bernardy, Patrik Jansson, and Koen Claessen. Testing Polymorphic Properties. *ESOP*, LNCS 6012, pp. 125–144, 2010. https://doi.org/10.1007/978-3-642-11957-6_8.
- [24] Mikołaj Bojańczyk, Bartek Klin, and Sławomir Lasota. Automata Theory in Nominal Sets. *Logical Methods in Computer Science* 10(3:4), 2014. [https://doi.org/10.2168/LMCS-10\(3:4\)2014](https://doi.org/10.2168/LMCS-10(3:4)2014). <https://arxiv.org/abs/1402.0897>.
- [25] Umut A. Acar, Matthias Blume, and Jacob Donham. A Consistent Semantics of Self-Adjusting Computation. Extended version, 2011. <https://arxiv.org/abs/1106.0478>.
- [26] Mario Carneiro. Lean4Lean: Towards a Verified Typechecker for Lean, in Lean. Preprint, 2024. <https://arxiv.org/abs/2403.14064>.
- [27] Anders Miltner, Adrian Trejo Nuñez, Ana Brendel, Swarat Chaudhuri, and Isil Dillig. Bottom-Up Synthesis of Recursive Functional Programs using Angelic Execution. *Proceedings of the ACM on Programming Languages* 6 (POPL), 2022. <https://doi.org/10.1145/3498682>. Project page: <https://www.cs.utexas.edu/~amiltner/burst/>.
- [28] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. Program Synthesis from Polymorphic Refinement Types. *PLDI*, 2016. <https://doi.org/10.1145/2908080.2908093>. <https://arxiv.org/abs/1510.08419>.
- [29] Márton Hajdú, Petra Hozzová, Laura Kovács, Johannes Schoisswohl, and Andrei Voronkov. Induction with Generalization in Superposition Reasoning. *CICM*, LNCS 12236, pp. 123–137, 2020. https://doi.org/10.1007/978-3-030-53518-6_8.

- [30] CoqHammer contributors. *CoqHammer: Proof Automation for Dependent Type Theory*. Official documentation, especially options for sauto. Consulted September 21, 2026. <https://coqhammer.github.io/>.
- [31] Derek Egolf and Stavros Tripakis. Recursive Program Synthesis from Sketches and Mixed-Quantifier Properties. Preprint, arXiv:2601.04045v2, July 22, 2026. <https://arxiv.org/html/2601.04045v2>.
- [32] Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. Premise Selection for a Lean Hammer. Preprint, 2025. <https://arxiv.org/abs/2506.07477>.
- [33] Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. LeanSearch v2: Global Premise Retrieval for Lean 4 Theorem Proving. Preprint, 2026. <https://arxiv.org/abs/2605.13137>.
- [34] Derek Egolf, Yuhao Zhou, and Stavros Tripakis. Can LLMs Perform Synthesis? Preprint, 2026. <https://arxiv.org/abs/2603.20264>.